MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 5 — Les Conditions

Apprendre à Python à prendre des décisions

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

3 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 5 — Les Conditions **2**
0.1 La structure if — La décision de base 2
0.2 if / else — Deux chemins possibles 2
0.3 if / elif / else — Plusieurs chemins 3
0.4 Les conditions imbriquées 4
0.5 Combiner les conditions avec and / or / not 5
0.6 L'opérateur ternaire — if en une ligne 5
0.7 Les valeurs truthy et falsy 6
0.8 match / case — Le nouveau switch de Python 3.10 7
0.9 Ce qui m'a le plus surpris dans ce chapitre 8
0.10 Résumé du Chapitre 5 8

Chapitre 5 — Les Conditions

À retenir

Les conditions sont le **cœur de la logique** d'un programme. Elles permettent à Python de prendre des décisions : faire une chose *si* une condition est vraie, une autre chose *sinon*. Sans conditions, un programme ne peut ni réagir, ni s'adapter, ni décider. C'est ici que le code devient **intelligent**.

La structure if — La décision de base

`if` est le mot-clé fondamental de toute condition. Il évalue une expression et exécute un bloc de code **uniquement si** cette expression est vraie.

Structure if — syntaxe de base

```python
# Syntaxe : if condition :
#                 bloc de code indente

age = 20

if age >= 18:
    print("Vous etes majeur.")
    print("Acces autorise.")

# Si la condition est False, le bloc est ignore
temperature = 10
if temperature > 30:
    print("Il fait chaud !")    # Non affiche
```

Erreur fréquente

L'**indentation** est OBLIGATOIRE en Python. Le bloc d'instructions sous un `if` doit être décalé de **4 espaces**. Si vous oubliez l'indentation ou le deux-points : après la condition, Python lève une `IndentationError` ou `SyntaxError` immédiatement.

if / else — Deux chemins possibles

`else` définit ce qui se passe quand la condition du `if` est **fausse**.

if / else — deux chemins

```python
age = 16

if age >= 18:
```

```
    print("Majeur     acces autorise.")
else:
    print("Mineur     acces refuse.")
#     Mineur     acces refuse.

# Exemple concret : verifier un mot de passe
mot_de_passe = input("Entrez le mot de passe : ")

if mot_de_passe == "Python2024":
    print("Connexion reussie !")
else:
    print("Mot de passe incorrect.")
```

if / elif / else — Plusieurs chemins

`elif` (contraction de *else if*) permet de tester **plusieurs conditions** en cascade. Python les évalue dans l'ordre et s'arrête dès qu'une est vraie.

if / elif / else — plusieurs conditions

```
note = 75

if note >= 90:
    mention = "Tres bien"
elif note >= 80:
    mention = "Bien"
elif note >= 70:
    mention = "Assez bien"
elif note >= 60:
    mention = "Passable"
else:
    mention = "Insuffisant"

print(f"Mention : {mention}")   # Mention : Assez bien

# Exemple : tarification selon l'age
age = int(input("Votre age : "))

if age < 4:
    prix = 0
    categorie = "Gratuit"
elif age < 12:
    prix = 5
    categorie = "Enfant"
elif age < 18:
    prix = 8
    categorie = "Adolescent"
elif age < 65:
    prix = 12
```

```
    categorie = "Adulte"
else:
    prix = 7
    categorie = "Senior"

print(f"Categorie : {categorie}     Prix : {prix}    ")
```

Ce qui m'a résisté

La subtilité avec `elif` : Python s'arrête à la **première condition vraie** et n'évalue pas les suivantes. Si j'écris `if note >= 60` en premier et `elif note >= 90` ensuite, une note de 95 déclenchera le premier bloc — pas le second. **L'ordre des conditions est crucial**. J'ai mis du temps à comprendre pourquoi mes résultats étaient faux, avant de réaliser que j'avais inversé l'ordre des seuils.

Les conditions imbriquées

On peut placer un `if` à l'intérieur d'un autre `if` pour créer des **logiques plus fines**.

Conditions imbriquées

```
age = 20
a_carte = True
est_vip = False

if age >= 18:
    print("Majeur confirme.")
    if a_carte:
        print("Carte verifiee.")
        if est_vip:
            print("Acces VIP     zone privee.")
        else:
            print("Acces standard     zone publique.")
    else:
        print("Carte requise pour entrer.")
else:
    print("Acces refuse     age insuffisant.")

#     Majeur confirme.
#     Carte verifiee.
#     Acces standard     zone publique.
```

Note importante

Les imbrications trop profondes rendent le code difficile à lire. En pratique, on évite de dépasser **3 niveaux** d'imbrication. Il vaut mieux utiliser `and` pour combiner les conditions quand c'est possible.

Combiner les conditions avec and / or / not

Plutôt que d'imbriquer, on peut combiner plusieurs conditions sur **une seule ligne**.

Conditions combinées

```python
age = 22
a_billet = True
est_interdit = False

# and     toutes les conditions doivent etre vraies
if age >= 18 and a_billet and not est_interdit:
    print("Entree autorisee.")

# or     au moins une condition doit etre vraie
jour = "samedi"
if jour == "samedi" or jour == "dimanche":
    print("C'est le week-end !")

# Exemple : validation d'un formulaire
prenom = input("Prenom : ")
age = int(input("Age : "))

if len(prenom) >= 2 and age >= 0 and age <= 120:
    print(f"Formulaire valide pour {prenom}, {age} ans.")
else:
    print("Donnees invalides.")
```

L'opérateur ternaire — if en une ligne

Python permet d'écrire une condition simple sur **une seule ligne** grâce à l'opérateur ternaire.

Opérateur ternaire

```python
# Syntaxe : valeur_si_vrai if condition else valeur_si_faux

age = 20
statut = "Majeur" if age >= 18 else "Mineur"
print(statut)   # Majeur

# Autres exemples
note = 55
resultat = "Recu" if note >= 60 else "Recale"
print(resultat)   # Recale

x = -5
valeur_absolue = x if x >= 0 else -x
print(valeur_absolue)   # 5
```

```python
# Dans un print directement
temperature = 35
print(f"Il fait {'chaud' if temperature > 25 else 'frais'} aujourd'hui.")
```

Ce qui m'a résisté

L'opérateur ternaire m'a d'abord semblé bizarre — l'ordre `valeur_si_vrai if condition else valeur_si_faux` est l'inverse de ce à quoi on pourrait s'attendre. On place d'abord **ce qu'on veut obtenir**, puis la condition. Il faut un moment pour que ça rentre naturellement, mais une fois maîtrisé, c'est redoutablement élégant pour simplifier le code.

Les valeurs truthy et falsy

En Python, toute valeur peut être évaluée comme `True` ou `False` dans une condition — pas seulement les booléens.

FALSY — évalué False	TRUTHY — évalué True
`False`	True
`0` et `0.0`	Tout entier non nul
`""` (chaîne vide)	Tout float non nul
`[]` (liste vide)	Chaîne non vide
`{}` (dict vide)	Liste non vide
`()` (tuple vide)	Dictionnaire non vide
`None`	Tout objet non None

Truthy et Falsy en pratique

```python
# On peut tester directement une valeur sans ==
prenom = ""
if prenom:
    print(f"Bonjour, {prenom} !")
else:
    print("Prenom vide     veuillez saisir votre prenom.")

# Tester si une liste est vide
panier = []
if panier:
    print(f"Vous avez {len(panier)} article(s).")
else:
    print("Votre panier est vide.")

# None est falsy
resultat = None
```

```python
if not resultat:
    print("Aucun resultat trouve.")
```

Ce qui m'a résisté

Les valeurs **truthy/falsy** m'ont vraiment surpris. Je ne savais pas qu'on pouvait écrire `if prenom:` au lieu de `if prenom != "":`. Au début ça me semblait moins clair — mais c'est en réalité plus **pythonique** et plus élégant. Python encourage à tester la valeur directement plutôt que de la comparer explicitement à zéro, None ou une chaîne vide.

match / case — Le nouveau switch de Python 3.10

Depuis Python 3.10, on dispose de `match / case` — l'équivalent moderne du *switch* que l'on trouve dans d'autres langages.

match / case — syntaxe moderne

```python
# Disponible depuis Python 3.10+
jour = "lundi"

match jour:
    case "lundi" | "mardi" | "mercredi" | "jeudi" | "vendredi":
        print("Jour de semaine     au travail !")
    case "samedi" | "dimanche":
        print("Week-end     repos merite !")
    case _:
        print("Jour inconnu.")

# Exemple avec des entiers
code = int(input("Code HTTP : "))

match code:
    case 200:
        print("OK     Succes")
    case 404:
        print("Not Found     Page introuvable")
    case 500:
        print("Internal Server Error")
    case _:
        print(f"Code {code} non reconnu")
```

Note importante

`match / case` n'est disponible qu'à partir de **Python 3.10**. Le `case _` joue le rôle du `else` — c'est le cas par défaut si aucun autre ne correspond. Pour des séries de comparaisons sur une même variable, `match` est beaucoup plus lisible qu'une longue

chaîne de `elif`.

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. L'indentation n'est pas optionnelle — c'est la syntaxe. Dans d'autres langages, les accolades `{}` délimitent les blocs. En Python, c'est l'indentation elle-même. Au début, ça m'a semblé fragile — une erreur d'espace et tout plante. Mais avec le temps, j'ai réalisé que ça **force** à écrire du code propre et structuré. C'est une contrainte qui devient une qualité.

2. Python s'arrête à la première condition vraie dans un elif. J'avais écrit mes conditions dans le mauvais ordre et mes résultats étaient systématiquement faux. Comprendre que `elif` est **exclusif** — une seule branche s'exécute, jamais deux — a été un déclic important. L'ordre des conditions n'est pas anodin. Il est stratégique.

3. Les valeurs truthy et falsy changent la façon d'écrire les conditions. Écrire `if liste:` au lieu de `if len(liste) > 0:` m'a semblé étrange puis libérateur. Python invite à penser différemment — pas en termes de comparaisons explicites, mais en termes de **présence** ou d'**absence** de valeur. C'est une philosophie à part entière.

Résumé du Chapitre 5

Ce que j'ai appris dans ce chapitre :

- `if / elif / else` — la structure de décision fondamentale de Python.
- L'**indentation** délimite les blocs — obligatoire, non négociable.
- `elif` s'arrête à la première condition vraie — l'ordre est stratégique.
- `and`, `or`, `not` — combiner les conditions élégamment.
- L'**opérateur ternaire** : `valeur_si_vrai if condition else valeur_si_faux`.
- Les valeurs **truthy/falsy** — toute valeur peut être testée directement.
- `match / case` — la syntaxe moderne de Python 3.10+ pour les cas multiples.

"La décision est le moteur de l'action. En programmation comme dans la vie — tout commence par un if."